2026

International Internship Pilot Program in Taiwan

Final Report

Name: Hilal İlhan

Internship Institution: National Taiwan University

Name of PI/Department/School: Prof. Cheng-Wei Chen / Next-generation Automated Surgical Apparatus Lab / Department of Electrical Engineering

Submission Date: September 2026

Combined ILC–MPC System

I. Introduction

This report presents the work carried out within the scope of my internship at the Next-generation Automated Surgical Apparatus Lab in the Department of Electrical Engineering at National Taiwan University under the supervision of Prof. Cheng-Wei Chen.

The purpose of the internship is to develop an understanding of Model Predictive Control (MPC) and Iterative Learning Control (ILC) and combine the key strengths of these two control algorithms while compensating for their individual limitations with the aim of improving reference tracking accuracy in dynamic systems. The outline of the internship includes the building of a theoretical background, the validation through simulation, and the evaluation of the results.

II. Objective

The main objective of this research is to achieve accurate tracking of a desired trajectory by combining two advanced control algorithms: Iterative Learning Control and Model Predictive Control.

In dynamic systems, violating the system constraints may create problems by requiring high input values and pushing the physical limits of the system. To avoid this, ILC is first used to repeatedly track a scaled-down version of the desired trajectory. This allows safe operation while collecting input–output data from the actual tracking process. Using this data, we learn the system's dynamics and acquire an estimated model of the system. Since the MPC algorithm highly depends on model accuracy to perform the optimization, the MPC block in our setup uses the estimated system model that is obtained by the ILC algorithm. MPC handles the constraints by optimizing the input over a receding horizon while respecting the system's limitations. This leads the controller to operate near the input limits without exceeding them. Thus, we obtain an attainable trajectory that actually takes the system's input and input-rate constraints into account. We feed this attained trajectory as a reshaped reference back to ILC and run the learning procedure again for the refinement of the estimated model.

Ultimately, we combine the learning capability of ILC with the constraint-handling capability of MPC and utilize the benefits of both algorithms. As a result, the system gradually achieves accurate tracking of a constraint-feasible version of the desired trajectory without violating the constraints by iteratively alternating between ILC-based learning and MPC-based reference shaping.

III. Theoretical Background

A. Iterative Learning Control (ILC)

The main objectives of studying Iterative Learning Control were to:

- understand how ILC improves reference tracking and reduces tracking error over successive trials;
- examine how the learning gain, number of trials, learning method, and Q-filter affect tracking performance; and
- investigate the limitation that conventional ILC does not directly account for input and input-rate constraints.

ILC is designed for systems that perform the same finite-duration task repeatedly. It uses the tracking error recorded during one trial to update the control input for the next trial. Through this trial-to-trial learning process, the system can progressively improve its tracking performance, even in the presence of model uncertainty and repeatable disturbances.

ILC is particularly useful for repetitive processes in which obtaining an exact system model is difficult. By using the known reference trajectory and error measurements from previous trials, it develops an improved feedforward control input over successive repetitions. This enables compensation in advance for errors and disturbances that occur consistently from one trial to another. Unlike feedback control, which reacts to the current tracking error in real time, ILC learns across repeated trials. Its general update structure is

$$\text{next input} = \text{previous input} + \text{learned correction}. \tag{1}$$

A general filtered ILC update law can be expressed as

$$u_{j+1}(k) = Q(q)\left[u_j(k) + L(q)e_j(k+1)\right], \tag{2}$$

where j denotes the trial number, k denotes the discrete-time index, $u_j(k)$ is the input, $e_j(k) = r(k) - y_j(k)$ is the tracking error, $L(q)$ is the learning operator, and $Q(q)$ is the robustness filter.

The Q-filter is generally implemented as a low-pass filter. It suppresses high-frequency components arising from measurement noise, modelling uncertainty, or aggressive learning updates while preserving the lower-frequency components associated with repeatable errors. Although stronger filtering can improve robustness, excessive filtering reduces the learning bandwidth and may limit the achievable tracking accuracy.

If the learning operator and Q-filter are selected appropriately, the tracking error decreases over successive trials. Aggressive learning gains, however, may produce trial-to-trial

oscillations or divergence. Conventional ILC also does not inherently enforce input or input-rate constraints, which motivates its combination with an optimization-based controller such as MPC.

B. Model Predictive Control (MPC)

The main objectives of studying Model Predictive Control were to:

- understand how MPC tracks a reference using a known state-space model of the plant;
- study how the prediction horizon, control horizon, cost-function weights, and constraints affect tracking performance; and
- compare tracking accuracy, control effort, and behavior under active constraints.

MPC predicts the future behavior of a system using a mathematical model and determines the control sequence by solving a constrained optimization problem. At each sampling instant, it uses the current state estimate to predict the system output over a finite horizon. It then selects the control sequence that minimizes the predicted tracking error and control effort while satisfying the specified constraints.

The discrete-time prediction model is

$$x(k+1) = Ax(k) + Bu(k), \tag{3}$$

$$y(k) = Cx(k) + Du(k). \tag{4}$$

The prediction horizon N_p determines how many future time steps are considered. A longer horizon allows MPC to anticipate upcoming reference changes and account for slower dynamics, but it also increases computational cost. The control horizon N_c determines how many future control moves can vary independently during each optimization. When $N_c < N_p$, the future input is typically held constant after the final optimized move.

Although an entire sequence of future inputs is calculated, only the first input is applied. At the next sampling instant, the state estimate is updated, the horizon moves forward, and the optimization is solved again. This procedure is known as the receding-horizon principle.

A simplified MPC optimization problem is

$$\min_{\Delta u(k),\ldots,\Delta u(k+N_c-1)} \quad J = \sum_{i=1}^{N_p} \|\hat{y}(k+i \mid k) - r(k+i)\|_Q^2 + \sum_{i=0}^{N_c-1} \|\Delta u(k+i \mid k)\|_R^2 , \tag{5}$$

subject to

$$u_{\min} \leq u(k+i \mid k) \leq u_{\max}, \tag{6}$$

$$\Delta u_{\min} \leq \Delta u(k+i \mid k) \leq \Delta u_{\max}. \tag{7}$$

Here, $\hat{y}(k+i \mid k)$ is the output predicted for time $k+i$ using information available at time k, $r(k+i)$ is the desired reference, and $\Delta u(k+i \mid k)$ is the predicted change in the control input. The weighting matrices Q and R determine the balance between tracking accuracy and smooth control action. The first element of the optimal sequence is applied as

$$u(k) = u(k-1) + \Delta u^*(k \mid k). \tag{8}$$

Classical MPC relies on the accuracy of the plant model and state estimate used to generate its predictions. Consequently, modelling errors, noisy system-identification data, disturbances, and state-estimation errors may reduce prediction accuracy and tracking performance.

C. Data-enabled Predictive Control (DeePC)

A data driven novel method in predictive control is used to improve the combined setup. DeePC utilizes the receding horizon optimization procedure similar to classical MPC. DeePC have also advantages compared to classical MPC such as being model-independent and just use collected data. Input output data is accumulated in Hankel matrix blocks. The procedure starts with collecting offline data, the input and output relation and matching trajectories. DeePC uses the superposition principle and optimizes over the weight vector g to make a linear combination of the trajectories matching their initial states as well. The Hankel matrix consists of past and future blocks, all coming from offline data collection step. So the ”future” block is actually already known and it is ”future ” relative to the initial standing state of the system.

IV. Approach and Methodology

To also understand the practical dimensions of ILC and MPC following my literature review, I built the simulations for ILC and MPC and ran them individually before combining the two algorithms. MATLAB is used for all of the simulations, as it offers extensive tools for control systems and their implementations. Later, we extended the simulation to run the two controllers in an alternating loop to achieve the objective of combining the key strengths of the algorithms.

A. Simulation Setup

The same plant and reference trajectory were used in all simulations to support a consistent comparison. The plant was defined as the continuous-time second-order system

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}, \tag{9}$$

with damping ratio $\zeta = 0.7$, natural frequency $\omega_n = 8$ rad/s, and unity DC gain. The plant was discretized using a zero-order hold with sampling time $T_s = 0.01$ s. It had a settling time of approximately 0.75 s, an overshoot of approximately 4.6%, and discrete poles at approximately

$$z = 0.944 \pm 0.054\text{j}. \tag{10}$$

The reference was a triangular signal ranging from zero to one, with a period of 4 s and an active duration of 12 s.

B. Standalone ILC Simulation

The standalone ILC simulation examined how tracking improved through repeated executions. The implementation did not contain an optimization problem, input saturation, or constraint-projection mechanism. It therefore learned the input required for tracking without guaranteeing satisfaction of the actuator limits.

1. PD-Type Learning Law

The initial simulation used the PD-type update

$$u_{j+1}(k) = Q(q)\left[u_j(k) + k_p e_j(k+1) + k_d\big(e_j(k+1) - e_j(k)\big)\right], \tag{11}$$

with

$$k_p = 0.266, \qquad k_d = 4. \tag{12}$$

The one-step-ahead error $e_j(k+1)$ is available because the complete previous-trial trajectory is stored before the next trial begins. Consequently, this noncausal update can be performed offline between trials.

2. Q-Filter

A 20th-order FIR low-pass Q-filter with a cutoff frequency of 2 Hz was applied after each update. Since $T_s = 0.01$ s, the sampling frequency was

$$f_s = \frac{1}{T_s} = 100 \text{ Hz}. \tag{13}$$

The cutoff was slightly higher than the approximate plant frequency $\omega_n/(2\pi) \approx 1.27$ Hz. MATLAB's `filtfilt` function applied the filter forward and backward, producing zero-phase offline filtering without phase delay.

3. Reference Padding

The active reference was padded with zeros at both ends. The number of padding samples on each side was selected from the plant settling time:

$$N_{\text{pad}} = \left\lceil \frac{T_{\text{settling}}}{T_s} \right\rceil = 75. \tag{14}$$

With 1200 active samples and 75 padding samples on each side, each trial contained

$$N = 1200 + 2(75) = 1350 \tag{15}$$

samples. The padding provided margin for forward–backward filtering transients and allowed the plant response to settle inside the trial window. Padded samples were excluded from the reported performance metrics.

4. Trial Execution and Performance Evaluation

The simulation consisted of 20 learning trials with zero initial input. MATLAB's `lsim` function generated the plant response, and the plant state was reset before every trial:

$$x_j(0) = 0, \qquad j = 1, \ldots, 20. \tag{16}$$

Tracking performance was evaluated over the active sample set $\mathcal{A}$ using

$$\text{RMSE}_j = \sqrt{\frac{1}{N_{\text{active}}} \sum_{k \in \mathcal{A}} e_j^2(k)}, \tag{17}$$

$$e_{\max,j} = \max_{k \in \mathcal{A}} |e_j(k)|. \tag{18}$$

The learned input was evaluated using its RMS magnitude, maximum magnitude, and maximum step-to-step change, where

$$\Delta u(k) = u(k) - u(k-1). \tag{19}$$

The input and input-rate values were compared with

$$|u(k)| \le 0.6, \qquad |\Delta u(k)| \le 0.01. \tag{20}$$

Since these limits were not included in the learning law, constraint satisfaction was evaluated rather than assumed.

C. Standalone MPC Simulation

The standalone MPC simulation evaluated reference tracking while directly enforcing the actuator constraints. Unlike the ILC simulation, it did not require reference padding because MPC operates causally and repeatedly solves its optimization problem during execution.

1. Controller Configuration

The controller was created using MATLAB's Model Predictive Control Toolbox. The prediction and control horizons were

$$N_p = 80, \qquad N_c = 20. \tag{21}$$

These values corresponded to prediction and control intervals of 0.80 s and 0.20 s, respectively. The prediction horizon therefore covered approximately one plant settling time.

The cost-function weights were

$$W_y = 20, \qquad W_u = 0.1, \qquad W_{\Delta u} = 0.1. \tag{22}$$

The comparatively large output weight prioritized tracking. The input weight discouraged unnecessarily large control actions, whereas the input-rate weight promoted smoother input changes.

The manipulated variable was constrained by

$$-0.6 \le u(k) \le 0.6, \tag{23}$$

and its step-to-step variation was constrained by

$$-0.01 \le \Delta u(k) \le 0.01. \tag{24}$$

Since the sampling time was 0.01 s, the input-rate limit corresponds to 1 input unit/s. These constraints were included directly in the MPC optimization problem.

2. Receding-Horizon Simulation

The closed-loop response was generated using MATLAB's `sim` function. Reference preview was enabled in the standalone case, allowing MPC to use upcoming reference values within its prediction horizon and anticipate changes in the triangular trajectory.

At each sampling instant, the controller estimated the current plant state, predicted the future output, solved the constrained optimization problem, and applied only the first element of the optimal input sequence. The optimization was then repeated at the next sample.

The simulation used the same nominal plant model for controller prediction and plant response. It therefore represented nominal MPC operation without model mismatch, measurement noise, or external disturbances.

3. Performance Evaluation

The tracking error was defined as $e(k) = r(k) - y(k)$. Tracking accuracy was evaluated using RMSE and maximum absolute error. Control demand and smoothness were evaluated using

$$u_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} u^2(k)}, \qquad u_{\mathrm{max,observed}} = \max_k |u(k)|, \tag{25}$$

$$\Delta u_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} \Delta u^2(k)}, \qquad \Delta u_{\mathrm{max,observed}} = \max_k |\Delta u(k)|. \tag{26}$$

These metrics demonstrate the trade-off between tracking accuracy, control effort, input smoothness, and constraint satisfaction. When a constraint becomes active, MPC may accept a larger tracking error to maintain a feasible control input.

D. Combined ILC–MPC Simulation

The combined framework was developed to exploit the complementary strengths of ILC and MPC. ILC-generated input–output data were used to identify a plant model. MPC then used this model to generate an attainable output trajectory under the prescribed actuator constraints. The MPC output became the reference for the following ILC stage, as shown in Figure 1.

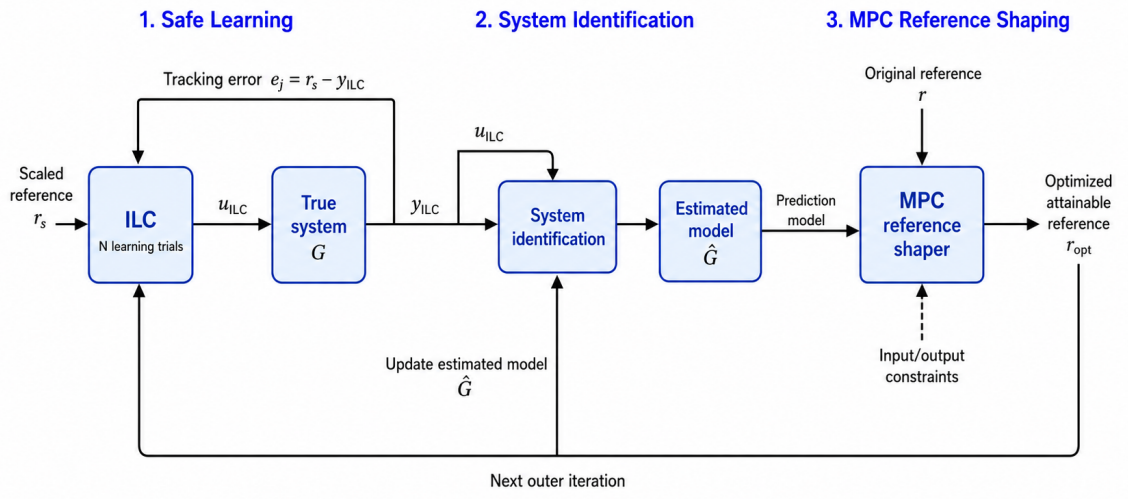


Fig 1. Block Diagram of the Combined ILC–MPC Loop

1. Inverse-Filter Learning for ILC

PD-type learning was first used in a preliminary identification phase to learn a fixed FIR inverse of the plant. During the main ILC trials, this inverse was used in place of the PD learning law to compensate for the plant dynamics more directly and track the references generated by MPC more efficiently.

Direct inversion over the entire frequency range can generate excessively large and noise-sensitive inputs. A band-limited impulse was therefore used as the inverse-identification reference:

$$r_{\text{inv}}(k) = \alpha M(q)\delta(k - k_c), \tag{27}$$

where $M(q)$ was a 20th-order FIR low-pass filter with a cutoff frequency of 2 Hz, k_c was the center of the trial, and $\alpha = 1$ was the impulse scale. Centering the impulse provided time-domain margin for both the causal and noncausal coefficients of the learned inverse.

The filtered impulse was tracked for ten trials using the PD update in Eq. (11). During this phase, the Q-filter was a 20th-order FIR low-pass filter with a cutoff frequency of 5 Hz. Zero-mean Gaussian measurement noise with standard deviation 0.01 was added to the output used by the learning update:

$$y_{\text{learn},j}(k) = y_{\text{true},j}(k) + v_j(k), \qquad v_j(k) \sim \mathcal{N}(0, 0.01^2). \tag{28}$$

After ten trials, the final learned input was normalized to obtain the fixed FIR inverse:

$$f_{\text{final}}(k) = \frac{u_{\text{inv,final}}(k)}{\alpha}. \tag{29}$$

For every subsequent ILC reference, this filter generated the initial feedforward input and the trial-to-trial corrections:

$$u_1 = F * r_{\text{ILC}}, \tag{30}$$

$$u_{j+1} = u_j + F * e_j, \tag{31}$$

where F represents f_{final}, and $*$ denotes centered FIR convolution. The centered convolution was implemented using MATLAB's `conv` function and aligned using the stored zero-lag index. Ten inverse-filter ILC trials were performed during every outer iteration, with the plant state reset to zero before each trial.

2. System Identification Using ILC Data

After the ILC trials in each outer iteration, the final learned input u_{ILC} and corresponding output y_{ILC} were stored as a discrete-time `iddata` experiment. Experiments from successive outer iterations were retained using `merge`:

$$\mathcal{D}_\ell = \left\{ (u_{\text{ILC}}^{(1)}, y_{\text{ILC}}^{(1)}), \ldots, (u_{\text{ILC}}^{(\ell)}, y_{\text{ILC}}^{(\ell)}) \right\}. \tag{32}$$

A stable second-order discrete-time state-space model with no direct feedthrough was estimated using the `ssest` function from the System Identification Toolbox in MATLAB:

$$\hat{x}(k+1) = \hat{A}\hat{x}(k) + \hat{B}u(k), \tag{33}$$

$$\hat{y}(k) = \hat{C}\hat{x}(k), \qquad \hat{D} = 0. \tag{34}$$

The estimated model was simplified using `minreal` to remove numerically redundant dynamics.

3. Constrained MPC Reference Generation

An MPC controller was constructed from the latest estimated model after every system-identification step. It used $N_p = 80$, $N_c = 20$, $W_y = 20$, $W_u = 0.1$, and $W_{\Delta u} = 0.1$, while enforcing

$$-0.6 \le u_{\text{MPC}}(k) \le 0.6, \qquad -0.01 \le \Delta u_{\text{MPC}}(k) \le 0.01. \tag{35}$$

MPC always attempted to track the original triangular reference. The resulting MPC output was defined as the feasible reference:

$$r^{(\ell)}_{\text{feasible}} = y^{(\ell)}_{\text{MPC}}. \tag{36}$$

Reference preview was disabled in the combined simulation. The feasible reference was therefore generated causally, without providing MPC with future changes in the desired trajectory.

4. Outer-Loop Reference Update

The combined procedure was repeated for five outer iterations. The scaling factor in the first iteration adjusted the reference so that the first ILC stage can track a scaled-down version of the original reference to obey the input constraint limits and ensure safety:

$$r^{(1)}_{\text{ILC}} = \alpha r_{\text{orig}}. \tag{37}$$

At the end of outer iteration ℓ, the MPC output became the ILC target for the following iteration:

$$r^{(\ell+1)}_{\text{ILC}} = y^{(\ell)}_{\text{MPC}}. \tag{38}$$

Accordingly, the combined algorithm consisted of the following steps:

1. Build the inverse filter by tracking a filtered impulse on the true plant using the PD-type learning law
2. Track the scaled down version of the original reference with ILC reference on the true plant using the fixed inverse filter.

3. Collect the final ILC input-output trajectory.

4. Add the trajectory to the accumulated system identification dataset.

5. Estimate a stable second-order model from all collected experiments using system identification block.

6. Track the original reference under the MPC input and input-rate constraints using the estimated model.

7. Use the MPC output as the ILC reference in the next outer iteration.

8. Collect new data from ILC and update the estimated model.

9. Repeat the procedure for five outer iterations.

5. Performance Evaluation

Two tracking-error metrics were used. The first quantified how accurately ILC tracked its current, potentially reshaped target:

$$\mathrm{RMSE}_{\mathrm{current}}^{(\ell)} = \sqrt{\frac{1}{N_{\mathrm{active}}} \sum_{k \in \mathcal{A}} \left[r_{\mathrm{ILC}}^{(\ell)}(k) - y_{\mathrm{ILC}}^{(\ell)}(k) \right]^2}. \tag{39}$$

The second quantified the difference between the ILC output and the original desired reference:

$$\mathrm{RMSE}_{\mathrm{original}}^{(\ell)} = \sqrt{\frac{1}{N_{\mathrm{active}}} \sum_{k \in \mathcal{A}} \left[r_{\mathrm{orig}}(k) - y_{\mathrm{ILC}}^{(\ell)}(k) \right]^2}. \tag{40}$$

This distinction is necessary because accurate tracking of a strongly reshaped reference does not necessarily imply accurate tracking of the original desired trajectory. The maximum ILC input and input-rate magnitudes were also recorded:

$$u_{\max}^{(\ell)} = \max_{k \in \mathcal{A}} |u_{\mathrm{ILC}}^{(\ell)}(k)|, \tag{41}$$

$$\Delta u_{\max}^{(\ell)} = \max_{k \in \mathcal{A}} |\Delta u_{\mathrm{ILC}}^{(\ell)}(k)|. \tag{42}$$

These quantities were compared with 0.6 and 0.01, respectively. The validation fit of the estimated model was also recorded at every outer iteration. Together, the metrics evaluate current-reference tracking, deviation from the original objective, actuator demand, constraint satisfaction, model quality, and changes across the outer loop.

V. Results and Discussion

The simulation results were evaluated by comparing the reference-tracking performance, required control input, and constraint-related behavior of standalone ILC, standalone MPC, and the combined ILC–MPC framework.

A. Standalone ILC Results

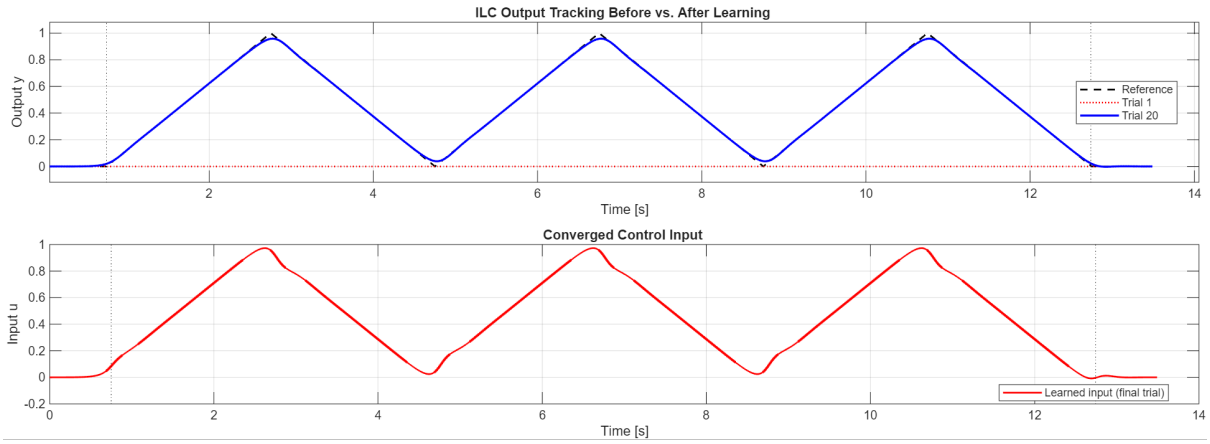


Fig 2. Results from the Standalone ILC Simulation

As shown in Figure 2, the output of the first ILC trial remains at zero because the control input is initialized to zero. The controller then uses the tracking error recorded during each trial to update the input for the following trial. By Trial 20, the system output follows the triangular reference closely over the complete active trajectory. Small deviations remain around the turning points of the reference because these regions require faster changes in the plant response and are affected by the bandwidth limitation introduced by the Q-filter.

The converged control input has a shape similar to the reference but includes additional corrections around the peaks and valleys to compensate for the plant dynamics. Although ILC achieves accurate tracking, the learned input reaches approximately 1. Therefore, it would exceed the upper input limit of 0.6 used in the constrained MPC and combined simulations. This result demonstrates the main limitation of standalone ILC: the algorithm minimizes repetitive tracking error but does not directly account for actuator input or input-rate constraints.

B. Standalone MPC Results

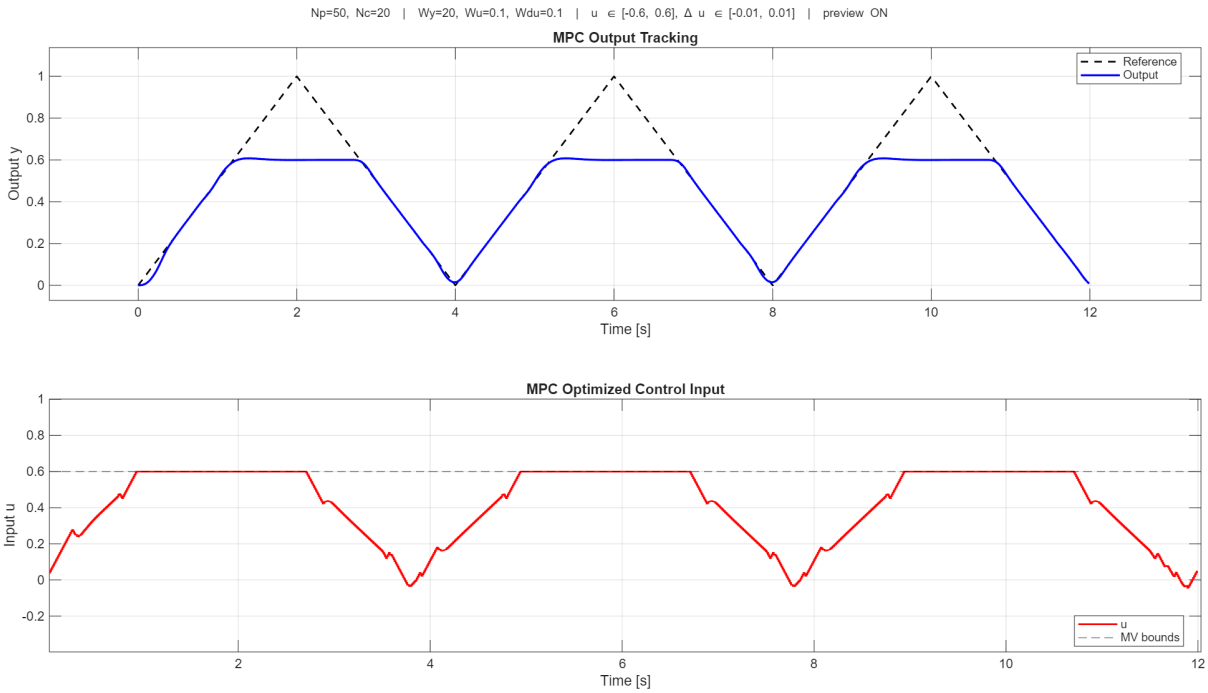


Fig 3. Results from the Standalone MPC Simulation

The standalone MPC results are presented in Figure 3. MPC follows the rising and falling sections of the triangular reference when the required control input remains feasible. However, the output cannot reach the reference peaks of 1. Instead, it remains close to 0.6 for part of each period, producing a flattened trajectory around the peaks.

The lower plot explains this behavior. The optimized MPC input reaches and remains at its upper bound of 0.6 during the corresponding time intervals. Once this constraint becomes active, the controller cannot further increase the input to reduce the remaining tracking error. MPC therefore sacrifices tracking accuracy to maintain a feasible control action. The input-rate constraint also limits how quickly the control input can change, which contributes to smoother control action but may introduce additional tracking lag.

These results show that the deviation from the original reference is not a failure of the MPC controller. Instead, it represents the closest trajectory that the controller can generate while considering its prediction model, cost-function weights, and imposed constraints. In contrast to standalone ILC, MPC explicitly incorporates these constraints into its optimization problem.

C. Combined ILC–MPC Results

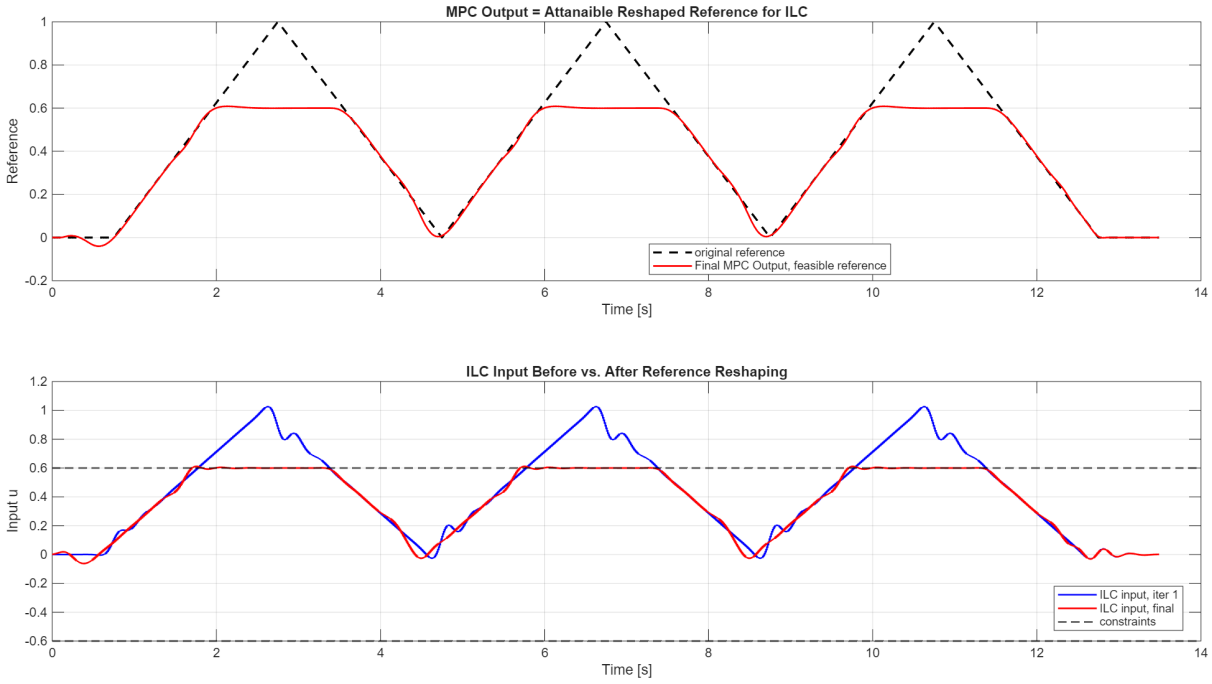


Fig 4. Results from the Combined ILC–MPC Simulation

Figure 4 shows how MPC reshapes the original triangular reference before it is passed to ILC. The MPC-generated reference follows the original trajectory while the required input remains feasible. Around the original peaks, it is limited to approximately 0.6, producing the flattened sections observed in the standalone MPC simulation. The resulting signal is therefore an attainable approximation of the original desired trajectory under the imposed constraints.

The lower plot compares the ILC input before and after reference reshaping. During the first outer iteration, the ILC input reaches values above 1, exceeding the upper bound of 0.6. After MPC reshapes the reference, the final ILC input is substantially reduced and remains close to or below the plotted magnitude limit. The greatest reduction occurs around the triangular peaks, where the original reference demanded the largest input.

The combined results demonstrate the complementary roles of the two controllers. ILC learns the feedforward input required to track a repetitive trajectory accurately, while MPC modifies the requested trajectory according to the system model and actuator constraints. Feeding the MPC output back to ILC reduces the control demand and allows ILC to focus on accurately tracking a more attainable reference.

VI. Conclusion

This study investigated the individual and combined performance of Iterative Learning Control and Model Predictive Control for repetitive reference tracking under actuator constraints. The standalone ILC simulation showed that trial-to-trial learning can considerably improve track-

ing accuracy. Nevertheless, because the learning law does not explicitly include constraints, the control input required to track the original reference exceeded the input limit used in the constrained simulations.

The standalone MPC controller successfully maintained the optimized input within the imposed bounds. However, when the input constraint became active, the controller could no longer follow the peaks of the original reference and produced a flattened output trajectory. This demonstrated the fundamental trade-off between reference-tracking accuracy and constraint satisfaction.

The combined ILC–MPC framework brought together the main advantages of both methods. ILC-generated data were used to obtain a model for MPC, and MPC used this model to reshape the original reference into an attainable trajectory. The reshaped reference was then supplied to ILC in the following outer iteration. As a result, the magnitude of the final learned ILC input was reduced from values above 1 to approximately the specified limit of 0.6.

Overall, the simulation results support the proposed combination: MPC provides constraint-aware reference reshaping, while ILC provides accurate repetitive tracking of the resulting trajectory. Further work should quantitatively evaluate input-rate satisfaction, tracking RMSE, model-estimation accuracy, and robustness under measurement noise and model mismatch. Experimental implementation on a physical system would also be necessary to verify whether the observed simulation behavior can be maintained under real actuator dynamics, disturbances, and measurement uncertainty.